

CMMS

Spis treści

Spis treści

1. Wstęp

- Opis Projektu
- Cel i przeznaczenie
- Zespół projektowy
- Zakres funkcjonalny
- Technologie

2. Wymagania

- Wymagania funkcjonalne
- Wymagania niefunkcjonalne
- Wymagania systemowe

3. Architektura

- Przegląd ogólny
- Diagram architektury
- Backend (apps/api)
- Frontend (apps/web)
- Infrastruktura (Docker)

4. Backend

- Technologie
- Warstwy Aplikacji
- Middleware
- Moduły
- Bezpieczeństwo

5. Frontend

- Technologie
- Struktura stron
- Zarządzanie sesją - AuthContext
- Kontrola dostępu - RoleGuard
- Komunikacja z API
- Komponenty współdzielone
- Stylowanie

6. Baza danych

- Przegląd
- Baza relacyjna - MaraiDB
- Moduł 1 — Użytkownicy**
- Moduł 2 — Lokalizacje i maszyny**
- Moduł 3 — Zlecenia pracy**

[Moduł 4 — Magazyn](#)

[Moduł 5 — Przeglądy prewencyjne](#)

[Moduł 6 — Powiadomienia](#)

[Baza Dokumentowa - MongoDB](#)

[Kolekcja `requestlogs`](#)

[Kolekcja `errorlogs`](#)

[Cache - Redis](#)

[7. API](#)

[Informacje ogólne](#)

[Endpointy](#)

[8. Testy](#)

[Technologie](#)

[Testy backendu](#)

[Testy frontendu](#)

[Uruchamianie testów](#)

[9. Instrukcja uruchomienia](#)

[Pobranie projektu](#)

[Uruchomienie aplikacji](#)

[Zatrzymanie aplikacji](#)

1. Wstęp

Opis Projektu

CMMS (ang. *Computerized Maintenance Management System*) to webowa aplikacja do zarządzania utrzymaniem ruchu w przedsiębiorstwie produkcyjnym. System powstał jako projekt zespołowy w ramach zajęć szkolnych i ma na celu zastąpienie tradycyjnych, papierowych metod zarządzania serwisem maszyn nowoczesnym, cyfrowym rozwiązaniem dostępnym przez przeglądarkę internetową.

Cel i przeznaczenie

Głównym celem aplikacji jest centralizacja informacji związanych z utrzymaniem maszyn i infrastruktury w jednym miejscu. System eliminuje konieczność prowadzenia papierowych dzienników serwisowych i arkuszy kalkulacyjnych, umożliwiając pracownikom różnych szczebli — od operatorów po administratorów — efektywne zarządzanie zleceniami naprawczymi, przeglądami technicznymi i magazynem części zamiennych.

Aplikacja skierowana jest do firm produkcyjnych, w których kluczową rolę odgrywa sprawność maszyn i ciągłość produkcji. Nieplanowane awarie generują wysokie koszty przestoju, dlatego system kładzie szczególny nacisk na planowanie przeglądów prewencyjnych oraz szybkie reagowanie na zgłoszone usterki.

Zespół projektowy

Projekt realizowany był przez trzyosobowy zespół:

Imię i nazwisko	Obszar odpowiedzialności
Patryk Stafecki	Backend - API, bazy danych, infrastruktura
Filip Lubka	Frontend - interfejs użytkownika
Jakub Owczarzak	Testy - testy jednostkowe backendu i frontendu, dokumentacja

Zakres funkcjonalny

System obejmuje następujące obszary:

- **Zarządzanie zleceniami pracy** — tworzenie, przypisywanie i śledzenie zleceń naprawczych wraz z potwierdzeniem procedur BHP
- **Przeglądy prewencyjne** — planowanie cyklicznych przeglądów maszyn z automatycznym generowaniem zleceń
- **Ewidencja maszyn** — dokumentacja techniczna z wersjonowaniem, licznik godzin pracy, raport kosztów (TCO)
- **Magazyn części zamiennych** — stany magazynowe, alerty niskiego stanu, wypożyczenia narzędzi
- **Zarządzanie użytkownikami** — konta pracowników, role dostępu, certyfikaty uprawnień
- **Powiadomienia** — powiadomienia systemowe i ogłoszenia masowe
- **Monitoring** — logi zapytań HTTP i błędów aplikacji
- **Dashboard** — podsumowanie KPI: MTTR, MTBF, koszty według maszyn, statystyki magazynu

Technologie

Warstwa	Technologia
Backend	Node.js, Hono.js, Prisma ORM, Zod
Frontend	Next.js 15, React 19, Sass, Recharts
Baza danych relacyjna	MariaDB
Baza danych dokumentowa	MongoDB
Cache / blacklista tokenów	Redis
Testy	Vitest, React Testing Library

Warstwa	Technologia
Infrastruktura	Docker, docker-compose

2. Wymagania

Wymagania funkcjonalne

Autoryzacja i użytkownicy

- System musi umożliwiać rejestrację i logowanie użytkowników
- Sesja użytkownika oparta jest na tokenach JWT z mechanizmem odświeżania
- Wylogowanie unieważnia token po stronie serwera (blacklista w Redis)
- Każdy użytkownik posiada przypisaną rolę określającą jego uprawnienia
- Administrator może tworzyć konta, zmieniać role i dezaktywować użytkowników
- System obsługuje certyfikaty uprawnień przypisane do użytkowników

Maszyny i lokalizacje

- System musi umożliwiać ewidencję maszyn z podstawowymi danymi technicznymi
- Każda maszyna przypisana jest do lokalizacji w hierarchicznej strukturze obiektu
- Do maszyny można dołączać dokumenty techniczne z automatycznym wersjonowaniem
- System rejestruje liczbę godzin pracy maszyny
- System generuje raport TCO (całkowity koszt posiadania) dla każdej maszyny

Zlecenia pracy

- Użytkownik z odpowiednią rolą może tworzyć zlecenia naprawcze
- Zlecenie przechodzi przez zdefiniowane stany: otwarte → w realizacji → zakończone
- Do zlecenia można przypisać serwisanta oraz użyte części magazynowe
- Serwisant musi potwierdzić zapoznanie się z procedurami BHP przed rozpoczęciem pracy
- Uczestnicy zlecenia mogą wymieniać wiadomości w ramach zlecenia

Przeglądy prewencyjne

- System umożliwia tworzenie planów przeglądów cyklicznych dla maszyn
- Plan zawiera checklistę czynności do wykonania
- Przeglądy mogą być zaplanowane według interwału czasowego (dni) lub godzin pracy maszyny
- System automatycznie generuje zlecenia pracy gdy nadejdzie termin przeglądu

- Możliwe jest ręczne wyzwolenie przeglądu poza harmonogramem

Magazyn

- System prowadzi ewidencję części zamiennych z kategoriami
- Stany magazynowe aktualizują się automatycznie przy użyciu części w zleceniach
- System generuje alert gdy stan magazynowy spadnie poniżej zdefiniowanego progu
- Użytkownicy mogą wypożyczać narzędzia z magazynu i je zwracać

Powiadomienia i monitoring

- System wysyła powiadomienia do użytkowników o zdarzeniach w systemie
- Administrator może wysyłać ogłoszenia do wszystkich aktywnych użytkowników
- System rejestruje każde zapytanie HTTP oraz błędy aplikacji
- Logi dostępne są dla administratora z możliwością filtrowania

Dashboard

- Po zalogowaniu użytkownik widzi panel z kluczowymi wskaźnikami (KPI)
- Dashboard prezentuje: liczbę otwartych zleceń, wskaźniki MTTR i MTBF, koszty według maszyn, statystyki magazynu i przeglądów

Wymagania niefunkcjonalne

Kategoria	Wymaganie
Bezpieczeństwo	Hasła przechowywane są wyłącznie w formie hashu (Argon2). Każdy endpoint wymaga uwierzytelnienia. Dostęp do zasobów kontrolowany jest przez role
Walidacja danych	Wszystkie dane wejściowe z żądań HTTP walidowane są przez schematy Zod przed przetworzeniem
Jakość kodu	Kod backendowy pokryty testami jednostkowymi (635 testów). Kod frontendowy pokryty testami jednostkowymi
Dostępność	Aplikacja dostępna przez przeglądarkę internetową bez instalacji dodatkowego oprogramowania po stronie użytkownika
Responsywność	Interfejs użytkownika dostosowany do różnych rozdzielczości ekranu

Wymagania systemowe

Środowisko uruchomieniowe (serwer)

Składnik	Minimalna wersja
Node.js	22.x lub nowszy
Docker	24.x lub nowszy

Składnik	Minimalna wersja
docker-compose	2.x lub nowszy

Bazy danych (uruchamiane przez Docker)

Baza danych	Zastosowanie
MariaDB	Dane relacyjne (użytkownicy, maszyny, zlecenia, magazyn)
MongoDB	Logi monitoringu (zapytania HTTP, błędy)
Redis	Blacklista unieważnionych tokenów JWT

Środowisko deweloperskie

Narzędzie	Zastosowanie
Node.js 22+	Uruchomienie aplikacji
npm	Zarządzanie zależnościami
Git	Kontrola wersji

Wymagania klienta (przeglądarka)

Aplikacja działa poprawnie w każdej nowoczesnej przeglądarce internetowej obsługującej JavaScript (Chrome, Firefox, Edge, Safari).

3. Architektura

Przegląd ogólny

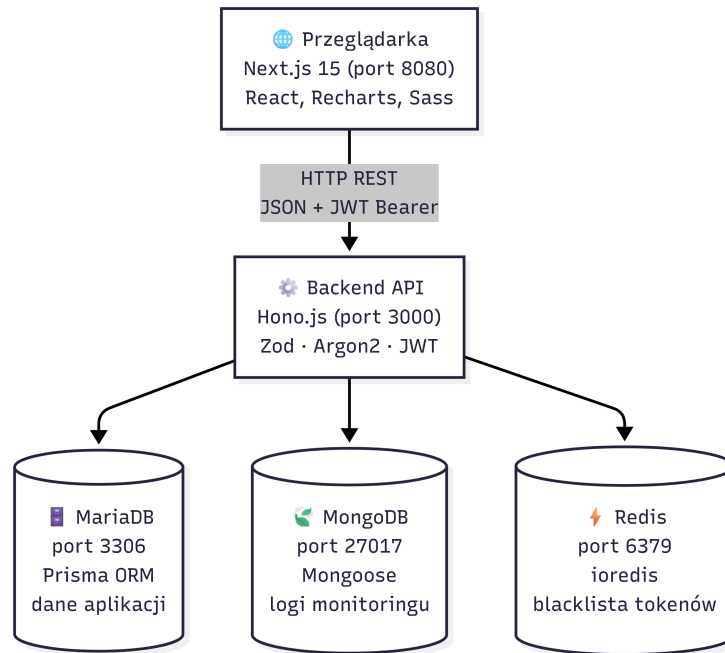
Aplikacja zbudowana jest jako **monorepo** — jedno repozytorium Git zawierające dwa niezależne pakiety uruchamiane osobno. Komunikacja między nimi odbywa się przez REST API po protokole HTTP.

```

cmms/
├── apps/
│   ├── api/    ← Backend (REST API, Node.js + Hono)
│   └── web/     ← Frontend (Next.js 15)
├── docker-compose.yml
└── package.json

```

Diagram architektury



Backend (apps/api)

• Struktura:

apps/api/src/

- |— app.ts ← Główny plik aplikacji, rejestracja routerów
- |— index.ts ← Punkt wejścia, uruchomienie serwera
- |— lib/
- | |— prisma.ts ← Klient Prisma (MariaDB)
- | |— mongoose.ts ← Połączenie MongoDB
- | |— redis.ts ← Klient Redis
- |— middleware/
- | |— auth.middleware.ts ← Weryfikacja tokenu JWT
- | |— roles.middleware.ts ← Kontrola dostępu (RBAC)
- | |— logger.middleware.ts ← Logowanie requestów i błędów
- |— models/
- | |— request-log.model.ts ← Model MongoDB (logi HTTP)
- | |— error-log.model.ts ← Model MongoDB (błędy)
- |— modules/
- |— auth/
- |— dashboard/
- |— inventory/
- |— locations/

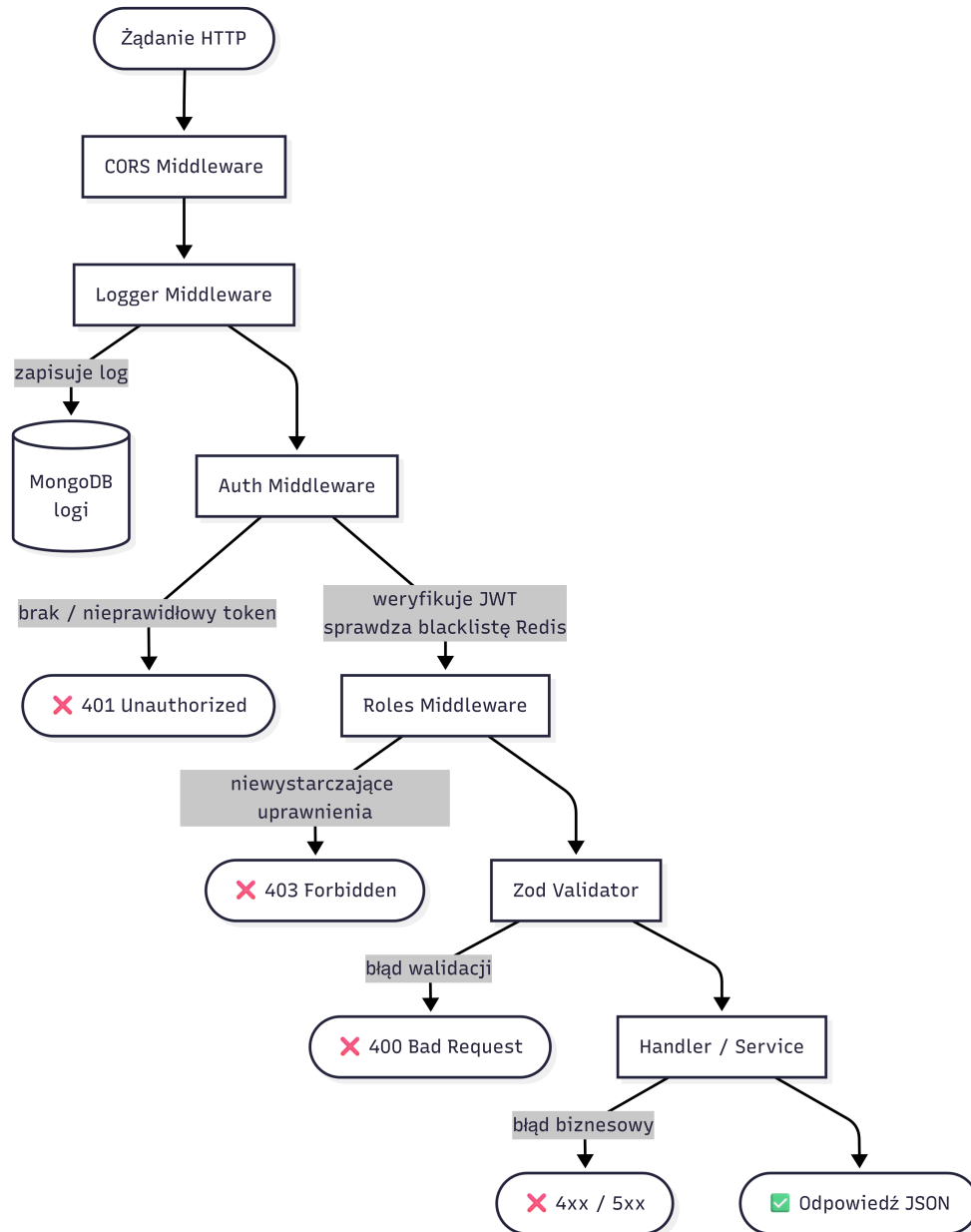
```
|— machines/  
|— monitoring/  
|— notifications/  
|— preventive/  
|— users/  
└— work-orders/
```

- Każdy moduł posiada jednolitą strukturę wewnętrzną:

modules/[nazwa]/

```
|— [nazwa].routes.ts ← Definicja endpointów HTTP  
|— [nazwa].service.ts ← Logika biznesowa, operacje na bazie  
|— [nazwa].schema.ts ← Schematy walidacji Zod  
└— tests/  
|— [nazwa].routes.test.ts  
└— [nazwa].service.test.ts
```

- Przepływ żądania HTTP

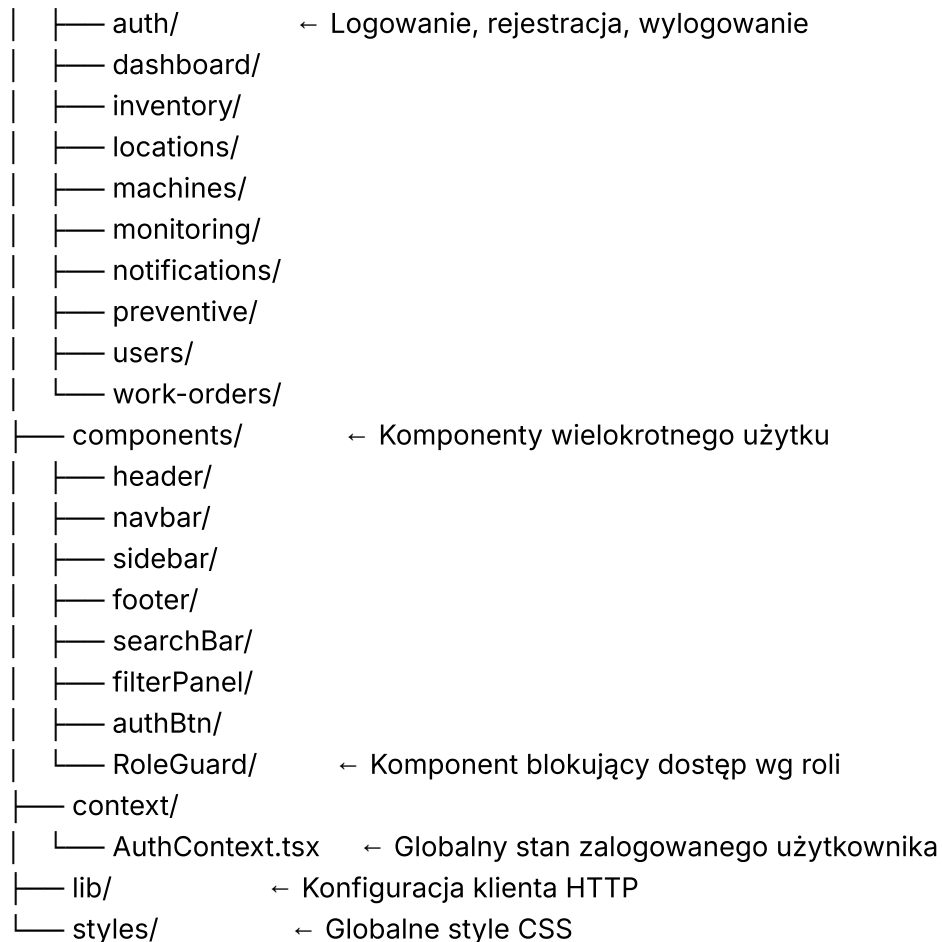


Frontend (apps/web)

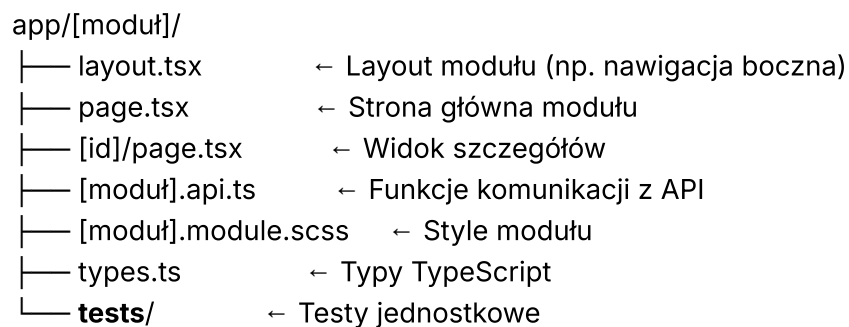
- Struktura:

apps/web/src/

— app/	← Strony (Next.js App Router)
— layout.tsx	← Główny layout z AuthProvider
— page.tsx	← Strona główna
— about/	



- Każdy moduł strony posiada typową strukturę:



Infrastruktura (Docker)

Kontener	Obraz	Port	Zastosowanie
cmms-mysql	mysql:8.0	3306	Główna baza danych relacyjnych
cmms-mongo	mongo:7	27017	Baza logów monitoringu
cmms-redis	redis:alpine	6379	Cache tokenów JWT

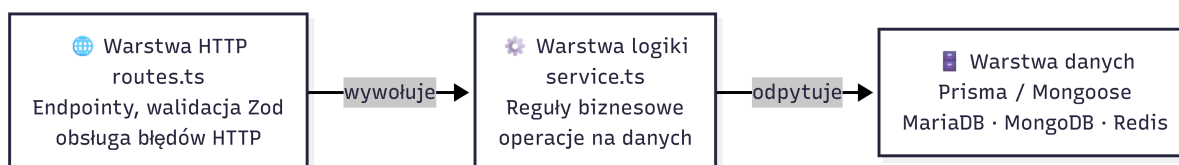
4. Backend

Technologie

Backend to REST API napisane w **TypeScript** na środowisku **Node.js**. Jako framework HTTP wykorzystano **Hono.js** — lekką bibliotekę zoptymalizowaną pod kątem wydajności, opartą na standardzie Web API (Request/Response).

Technologia	Wersja	Zastosowanie
Node.js	22+	Środowisko uruchomieniowe
Hono.js	4.x	Framework HTTP
Prisma ORM	7.x	Komunikacja z MariaDB
Mongoose	9.x	Komunikacja z MongoDB
iodis	5.x	Komunikacja z Redis
Zod	4.x	Walidacja danych wejściowych
Argon2	2.x	Hashowanie haseł
jose	6.x	Generowanie i weryfikacja JWT
node-cron	4.x	Harmonogram zadań cyklicznych

Warstwy Aplikacji



Routes — przyjmuje żądanie HTTP, waliduje dane wejściowe przez Zod, wywołuje odpowiednią funkcję serwisu i zwraca odpowiedź JSON.

Service — zawiera wyłącznie logikę biznesową: sprawdza reguły domenowe, wykonuje operacje na bazie danych przez Prisma lub Mongoose, rzuca wyjątki `HTTPException` w przypadku błędów.

Schema — definicje schematów Zod używanych zarówno do walidacji jak i do generowania typów TypeScript.

Middleware

Każde żądanie HTTP przechodzi przez łańcuch middleware zanim trafi do handlera modułu.

- `auth.middleware.ts` — **uwierzytelnianie**

Weryfikuje token JWT z nagłówka `Authorization: Bearer <token>`. Sprawdza czy token nie został unieważniony (blacklista w Redis). W przypadku powodzenia umieszcza dane użytkownika (`id`, `email`, `role`) w kontekście żądania.

- `roles.middleware.ts` — **autoryzacja**

Sprawdza czy zalogowany użytkownik posiada wymaganą rolę do wykonania danej operacji. Jeśli nie — zwraca `403 Forbidden` z informacją o wymaganych rolach.

- `logger.middleware.ts` — **logowanie**

Rejestruje w MongoDB żądania HTTP spełniające co najmniej jeden z warunków:

- kod odpowiedzi `≥ 400` (błędy)
- czas odpowiedzi `> 1000ms` (wolne zapytania)
- ścieżka zawiera `/auth/login` (próby logowania)

Błędy nieobsłużone przez aplikację są przechwytywane przez `errorLoggerMiddleware` — zapisywane do MongoDB z pełnym stack trace i zwracane klientowi jako JSON.

Moduły

Funkcja	Opis
<code>register</code>	Tworzy nowe konto, hashuje hasło Argon2, zwraca tokeny
<code>login</code>	Weryfikuje hasło, generuje access token (15 min) i refresh token (7 dni)
<code>logout</code>	Dodaje token do blacklisty w Redis
<code>refreshTokens</code>	Wymienia refresh token na nową parę tokenów, stary trafia na blacklistę
<code>getMe</code>	Zwraca dane zalogowanego użytkownika wraz z certyfikatami

- `users` — **użytkownicy**

Zarządzanie kontami pracowników. Operacje dostępne wyłącznie dla ról ADMIN i MANAGER. Obsługuje paginację, filtrowanie po roli i statusie aktywności. Zawiera zabezpieczenie przed usunięciem ostatniego aktywnego administratora.

- `machines` — **maszyny**

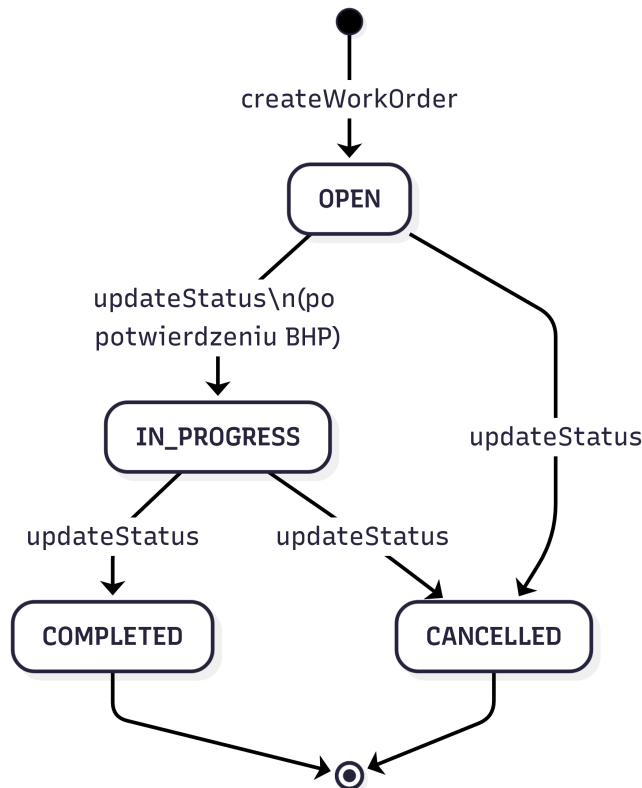
Ewidencja maszyn produkcyjnych. Kluczowe funkcje: wersjonowanie dokumentów technicznych (każdy upload automatycznie inkrementuje numer wersji), aktualizacja licznika godzin pracy, generowanie raportu TCO agregującego koszty robocizny i części ze wszystkich zakończonych zleceń.

- `locations` — **lokalizacje**

Hierarchiczna struktura obiektu (np. hala → strefa → stanowisko). Lokalizacja może posiadać lokalizacje nadrzędne i podrzędne. System blokuje usunięcie lokalizacji, do której przypisane są maszyny lub lokalizacje potomne.

- **work-orders** — **zlecenia pracy**

Najobszerniejszy moduł systemu. Obsługuje pełny cykl życia zlecenia naprawczego:



Zlecenie wymaga potwierdzenia BHP przez przypisanego serwisanta przed zmianą stanu na **IN_PROGRESS**. Do zlecenia można dołączać części z magazynu (automatycznie dekrementując stan magazynowy) oraz wymieniać wiadomości.

- **preventive** — **przeglądy prewencyjne**

Zarządza planami cyklicznych przeglądów maszyn. Każdy plan zawiera checklistę czynności i interwał (w dniach lub godzinach pracy). Co godzinę uruchamiany jest harmonogram cron:

```
cron('0 * * * *') → checkAndCreatePreventiveOrders()
```

Funkcja sprawdza wszystkie aktywne plany i automatycznie tworzy zlecenia pracy dla tych, których termin (`nextRunAt`) minął.

- **inventory** — magazyn

Zarządza częściami zamiennymi i narzędziami. Po każdej korekcie stanu magazynowego sprawdzany jest próg reorder point — jeśli stan spada poniżej progu, automatycznie tworzone jest powiadomienie `REORDER_ALERT` dla menedżerów. Moduł obsługuje również wypożyczanie narzędzi z automatyczną dekrementacją stanu przy wypożyczeniu i inkrementacją przy zwrocie.

- **notifications** — powiadomienia

Obsługuje powiadomienia indywidualne (dla jednego użytkownika) oraz ogłoszenia masowe (dla wszystkich aktywnych użytkowników). Powiadomienia oznaczane są jako przeczytane, a ich licznik nieprzeczytanych dostępny jest przez dedykowany endpoint.

- **dashboard** — panel główny

Agreguje dane z całego systemu i oblicza kluczowe wskaźniki operacyjne:

Wskaźnik	Opis
MTTR	Średni czas naprawy (Mean Time To Repair) — średni czas od otwarcia do zamknięcia zlecenia
MTBF	Średni czas między awariami (Mean Time Between Failures) — średni czas między kolejnymi zleceniami dla tej samej maszyny
Koszty	Zestawienie kosztów robocizny i części według maszyn (top 5)
Magazyn	Liczba części, alerty niskiego stanu, aktywne wypożyczenia
Przeglądy	Liczba planów, nadchodzące i przeterminowane

- **monitoring** — logi systemowe

Dostępny wyłącznie dla roli ADMIN. Udostępnia logi requestów HTTP i błędów aplikacji przechowywanych w MongoDB z możliwością filtrowania po użytkowniku, kodzie odpowiedzi i zakresie dat.

Bezpieczeństwo

Mechanizm	Implementacja
Hashowanie haseł	Argon2 — odporny na ataki słownikowe i brute-force
Uwierzytelnianie	JWT — access token (15 min), refresh token (7 dni)
Unieważnianie tokenów	Blacklista w Redis z TTL równym pozostałemu czasowi ważności tokenu
Autoryzacja	RBAC — każdy endpoint deklaruje wymagane role

Mechanizm	Implementacja
Walidacja wejścia	Zod — wszystkie dane wejściowe walidowane przed przetworzeniem
CORS	Ograniczony do adresu frontendu (<code>localhost:8080</code>)

5. Frontend

Technologie

Frontend to aplikacja webowa zbudowana w **Next.js 15** z wykorzystaniem **App Router** — systemu routingu opartego na strukturze katalogów. Aplikacja działa w trybie klienckim (`'use client'`) dla stron wymagających interaktywności i komunikacji z API.

Technologia	Wersja	Zastosowanie
Next.js	15.x	Framework React, routing, SSR
React	19.x	Biblioteka interfejsu użytkownika
Recharts	3.x	Wykresy na dashboardzie
Sass	1.x	Stylowanie komponentów (CSS Modules)
js-cookie	3.x	Obsługa cookies (tokeny JWT)

Struktura stron

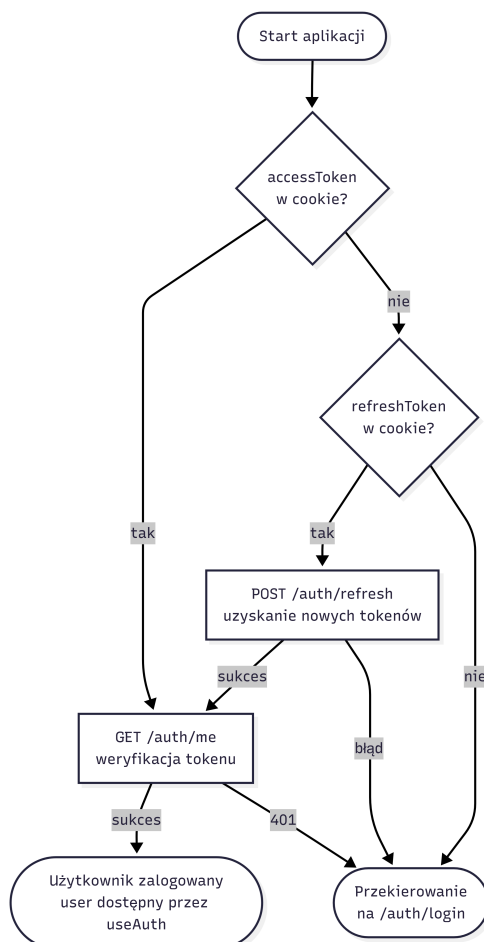
Aplikacja korzysta z **Next.js App Router** — każdy folder w katalogu `app/` odpowiada ścieżce URL:

Ścieżka URL	Opis
<code>/</code>	Strona główna ze sliderem
<code>/about</code>	Strona „O nas” z animacją
<code>/auth/login</code>	Logowanie
<code>/auth/register</code>	Rejestracja
<code>/auth/logout</code>	Wylogowanie
<code>/dashboard</code>	Panel główny z KPI
<code>/inventory</code>	Magazyn części zamiennych
<code>/locations</code>	Lokalizacje
<code>/locations/[id]</code>	Szczegóły lokalizacji
<code>/machines</code>	Lista maszyn
<code>/machines/[id]</code>	Szczegóły maszyny
<code>/monitoring</code>	Logi systemowe

Ścieżka URL	Opis
/notifications	Powiadomienia
/preventive	Przeglądy prewencyjne
/users	Użytkownicy
/work-orders	Zlecenia pracy
/work-orders/[id]	Szczegóły zlecenia

Zarządzanie sesją - AuthContext

Globalny stan sesji użytkownika zarządzany jest przez `AuthContext` dostępny w całej aplikacji przez hook `useAuth()`.



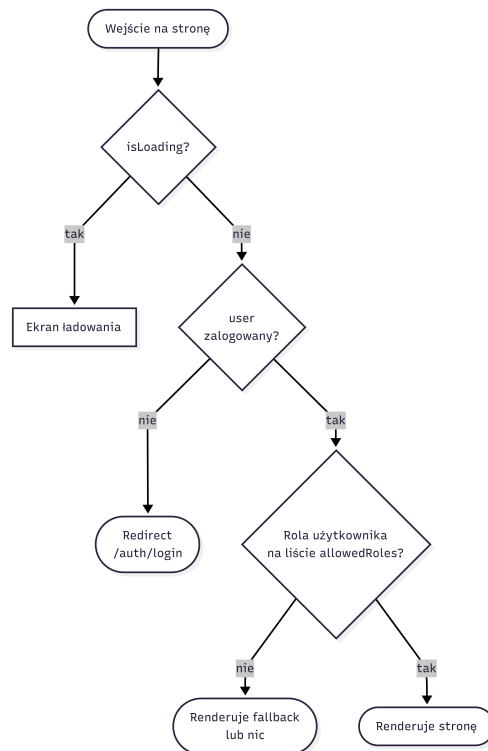
Dane użytkownika (`id`, `name`, `email`, `role`) przechowywane są w stanie React i dostępne przez hook `useAuth()` w każdym komponencie. Tokeny JWT przechowywane są w cookies z

flagami `secure` i `sameSite: strict`.

Kontrola dostępu - RoleGuard

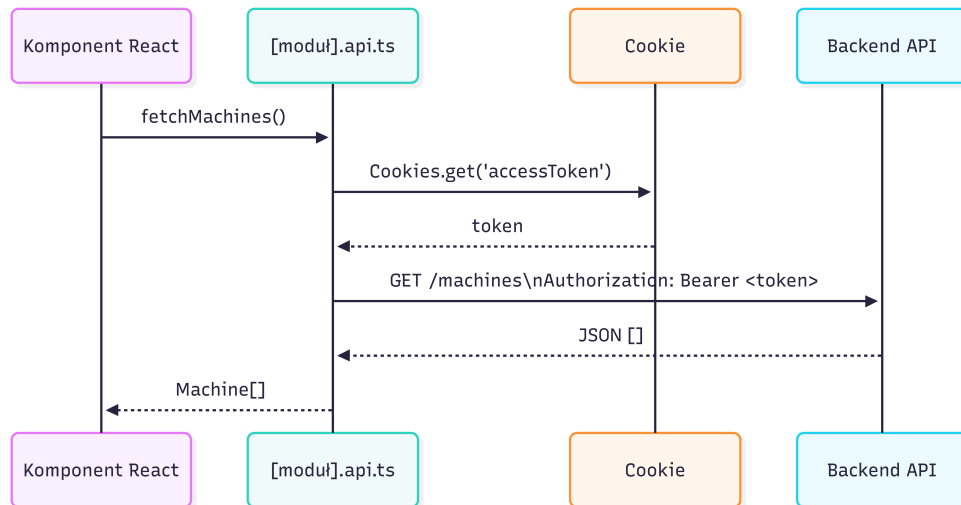
Każda chroniona strona opakowana jest komponentem `<RoleGuard>`, który:

1. Sprawdza czy użytkownik jest zalogowany — jeśli nie, przekierowuje na `/auth/login`
2. Sprawdza czy rola użytkownika jest na liście dozwolonych — jeśli nie, renderuje `fallback` (komunikat o braku dostępu) lub nic



Komunikacja z API

Każdy moduł strony posiada dedykowany plik `[moduł].api.ts` zawierający funkcje komunikacji z backendem. Token JWT dołączany jest automatycznie do każdego żądania:



Komponenty współdzielone

Wielokrotnie używane elementy interfejsu wydzielone zostały do katalogu `src/components/`:

Komponent	Opis
<code>Header</code>	Górny pasek z logo, linkiem do dashboardu i przyciskiem auth
<code>Navbar</code>	Boczny panel nawigacyjny z linkami do modułów
<code>Sidebar</code>	Wysuwane menu mobilne
<code>Footer</code>	Stopka strony
<code>SearchBar</code>	Pole wyszukiwania z debounce
<code>FilterPanel</code>	Panel filtrów (np. po roli, statusie)
<code>AuthBtn</code>	Przycisk logowania / wylogowania w zależności od stanu sesji
<code>RoleGuard</code>	Komponent ochrony dostępu oparty na roli

Stylowanie

Aplikacja korzysta z **CSS Modules** w połączeniu z **Sass** (`.module.scss`). Każdy moduł strony i komponent posiada własny plik stylów, co eliminuje konflikty nazw klas i zapewnia izolację stylów. Globalne zmienne CSS (kolory, fonty, odstępy) zdefiniowane są w pliku `styles/`.

6. Baza danych

Przegląd

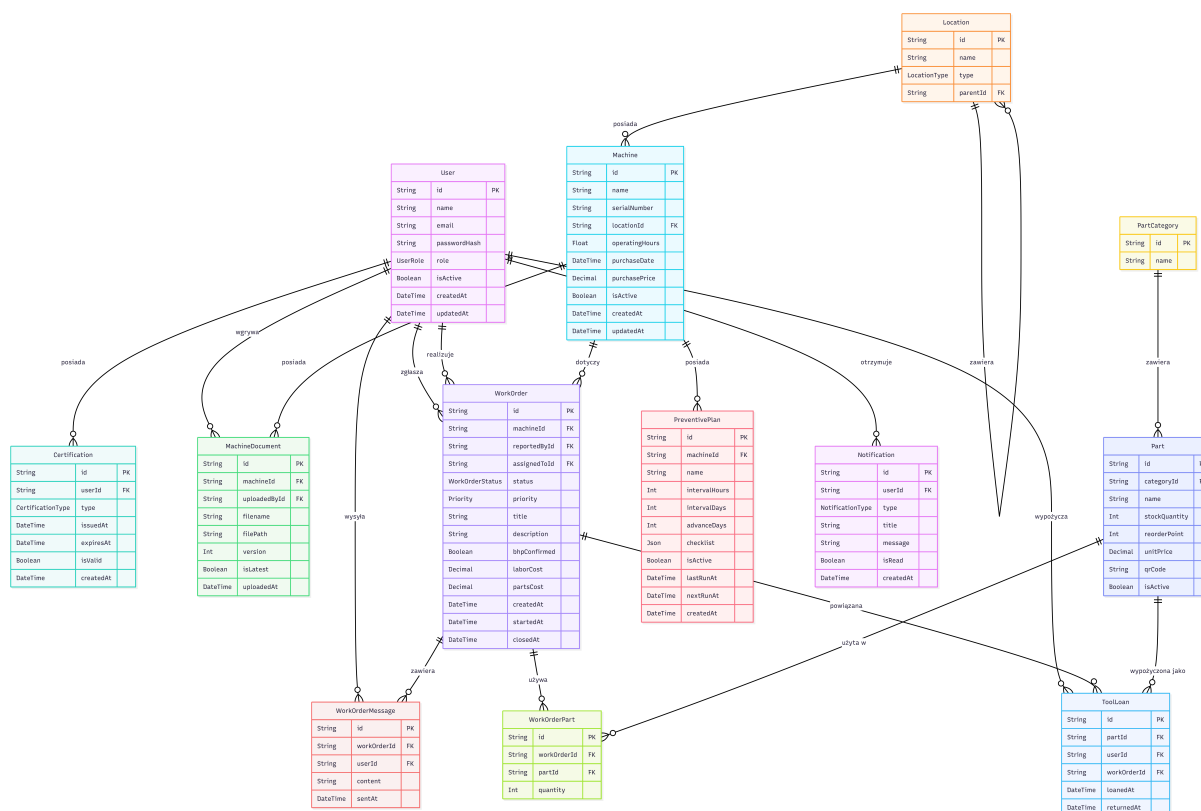
Projekt wykorzystuje dwa różne silniki baz danych, każdy dobrany do charakteru przechowywanych danych:

Baza danych	Silnik	ORM / Driver	Zastosowanie
Relacyjna	MariaDB	Prisma ORM	Dane aplikacji (użytkownicy, maszyny, zlecenia, magazyn)
Dokumentowa	MongoDB	Mongoose	Logi monitoringu (requesty HTTP, błędy aplikacji)
Cache	Redis	ioredis	Blacklista unieważnionych tokenów JWT

Baza relacyjna - MariaDB

Główna baza danych przechowuje wszystkie dane biznesowe aplikacji. Schemat zdefiniowany jest w Prisma (`schema.prisma`) i podzielony na 6 modułów tematycznych.

• Diagram ERD



• Opis tabel

Moduł 1 — Użytkownicy

users — konta pracowników systemu. Każdy użytkownik posiada przypisaną rolę (**ADMIN** , **MANAGER** , **TECHNICIAN** , **WAREHOUSE** , **OPERATOR**) oraz flagę **isActive** umożliwiającą dezaktywację konta bez usuwania danych.

certifications — certyfikaty uprawnień przypisane do użytkowników (np. uprawnienia elektryczne SEP, obsługa wózków widłowych). Każdy certyfikat posiada datę wystawienia i wygaśnięcia.

Moduł 2 — Lokalizacje i maszyny

locations — hierarchiczna struktura obiektu. Tabela odwołuje się sama do siebie przez **parentId** — lokalizacja może mieć lokalizację nadrzędną, tworząc drzewo (zakład → hala → linia → stanowisko).

machines — ewidencja maszyn produkcyjnych z danymi technicznymi i finansowymi. Powiązana z lokalizacją przez **locationId**. Flaga **isActive** umożliwia soft-delete.

machine_documents — dokumenty techniczne maszyn z wersjonowaniem. Pole **version** inkrementowane jest przy każdym wgraniu nowego pliku. Flaga **isLatest** wskazuje aktualną wersję.

Moduł 3 — Zlecenia pracy

work_orders — zlecenia naprawcze. Posiada dwa klucze obce do tabeli **users** : **reportedById** (kto zgłosił) i **assignedToId** (kto realizuje). Pola **startedAt** i **closedAt** umożliwiają obliczenie MTTR. Flaga **bhpConfirmed** blokuje zmianę statusu do czasu potwierdzenia procedur BHP.

work_order_messages — wiadomości wymieniane przez uczestników zlecenia.

work_order_parts — tabela łącząca zlecenia z używanymi częściami (relacja wiele-do-wielu). Przechowuje zużytą ilość.

Moduł 4 — Magazyn

part_categories — kategorie części zamiennych.

parts — części zamienne z bieżącym stanem magazynowym (**stockQuantity**) i progiem ponownego zamówienia (**reorderPoint**). Każda część może mieć unikalny kod QR. Soft-delete przez **isActive** .

tool_loans — wypożyczenia narzędzi przez pracowników. Pole **returnedAt** przyjmuje wartość **NULL** dla aktywnych wypożyczeń.

Moduł 5 — Przeglądy prewencyjne

preventive_plans — plany cyklicznych przeglądów maszyn. Interwał określony w dniach (**intervalDays**) lub godzinach pracy maszyny (**intervalHours**). Pole **checklist** przechowuje listę czynności jako JSON. Pola **lastRunAt** i **nextRunAt** zarządzane są automatycznie przez harmonogram cron.

Moduł 6 — Powiadomienia

`notifications` — powiadomienia systemowe dla użytkowników.

Typy: `CRITICAL_FAILURE`, `REORDER_ALERT`, `PREVENTIVE_DUE`, `CERT_EXPIRING`, `WORK_ORDER_ASSIGNED`, `ANNOUNCEMENT`.

Baza Dokumentowa - MongoDB

MongoDB przechowuje logi generowane przez middleware monitoringu. Dane logów mają charakter append-only (tylko dopisywanie) i nie wymagają relacji — stąd wybór bazy dokumentowej.

Kolekcja `requestlogs`

Rejestruje wybrane żądania HTTP (błędy, wolne zapytania, logowania):

Pole	Typ	Opis
<code>method</code>	String	Metoda HTTP (GET, POST, ...)
<code>url</code>	String	Pełny adres URL żądania
<code>statusCode</code>	Number	Kod odpowiedzi HTTP
<code>duration</code>	Number	Czas odpowiedzi w ms
<code>userId</code>	String	ID zalogowanego użytkownika (opcjonalne)
<code>userRole</code>	String	Rola użytkownika
<code>ip</code>	String	Adres IP klienta
<code>userAgent</code>	String	Identyfikator przeglądarki
<code>createdAt</code>	Date	Znacznik czasu

Kolekcja `errorlogs`

Rejestruje nieobsłużone błędy aplikacji:

Pole	Typ	Opis
<code>message</code>	String	Treść błędu
<code>stack</code>	String	Stack trace
<code>statusCode</code>	Number	Kod HTTP błędu
<code>method</code>	String	Metoda HTTP żądania
<code>url</code>	String	Adres URL żądania
<code>createdAt</code>	Date	Znacznik czasu

Cache - Redis

Redis pełni funkcję szybkiej pamięci podręcznej do przechowywania blacklisty unieważnionych tokenów JWT. Po wylogowaniu użytkownika token zapisywany jest pod kluczem `blacklist:<jti>` z TTL równym pozostałemu czasowi ważności tokenu — po jego upływie wpis automatycznie znika z Redis.

7. API

Informacje ogólne

Backend udostępnia REST API nasłuchujące na porcie **3000**. Wszystkie odpowiedzi zwracane są w formacie **JSON**. Większość endpointów wymaga uwierzytelnienia — token JWT przekazywany jest w nagłówku:

```
Authorization: Bearer <accessToken>
```

Błędy zwracane są w ujednoliconym formacie

```
{ "message": "Opis błędu" }
```

• Kody odpowiedzi HTTP

Kod	Znaczenie
200	Sukces
201	Zasób utworzony
400	Błąd walidacji danych wejściowych
401	Brak lub nieprawidłowy token
403	Niewystarczające uprawnienia (rola)
404	Zasób nie istnieje
409	Konflikt (np. duplikat)
500	Błąd wewnętrzny serwera

Endpointy

• `auth` — autoryzacja

Metoda	Ścieżka	Opis	Auth
POST	<code>/auth/register</code>	Rejestracja nowego użytkownika	✗

Metoda	Ścieżka	Opis	Auth
POST	/auth/login	Logowanie, zwraca parę tokenów	✗
POST	/auth/logout	Wylogowanie, unieważnia token	✓
POST	/auth/refresh	Odświeżenie tokenów	✗
GET	/auth/me	Dane zalogowanego użytkownika	✓

- users** — użytkownicy

Metoda	Ścieżka	Opis	Role
GET	/users	Lista użytkowników (paginacja, filtry)	ADMIN, MANAGER
GET	/users/:id	Szczegóły użytkownika	ADMIN, MANAGER
PATCH	/users/:id	Edycja danych użytkownika	ADMIN
DELETE	/users/:id	Dezaktywacja użytkownika	ADMIN
GET	/users/:id/certifications	Lista certyfikatów użytkownika	ADMIN, MANAGER
POST	/users/:id/certifications	Dodanie certyfikatu	ADMIN
DELETE	/users/:id/certifications/:certId	Usunięcie certyfikatu	ADMIN

- locations** — lokalizacje

Metoda	Ścieżka	Opis	Role
GET	/locations	Lista lokalizacji z hierarchią	ALL
GET	/locations/:id	Szczegóły lokalizacji z dziećmi	ALL
POST	/locations	Dodanie lokalizacji	ADMIN, MANAGER
PATCH	/locations/:id	Edycja lokalizacji	ADMIN, MANAGER
DELETE	/locations/:id	Usunięcie lokalizacji	ADMIN

- machines** — maszyny

Metoda	Ścieżka	Opis	Role
GET	/machines	Lista aktywnych maszyn	ALL
GET	/machines/:id	Szczegóły maszyny	ALL
POST	/machines	Dodanie maszyny	ADMIN, MANAGER
PATCH	/machines/:id	Edycja maszyny	ADMIN, MANAGER
DELETE	/machines/:id	Usunięcie maszyny	ADMIN

Metoda	Ścieżka	Opis	Role
PATCH	/machines/:id/operating-hours	Aktualizacja godzin pracy	ADMIN, MANAGER, TECHNICIAN
GET	/machines/:id/documents	Lista dokumentów maszyny	ALL
POST	/machines/:id/documents	Wgranie dokumentu	ADMIN, MANAGER
GET	/machines/:id/tco	Raport TCO maszyny	ADMIN, MANAGER

- work-orders — zlecenia pracy**

Metoda	Ścieżka	Opis	Role
GET	/work-orders	Lista zleceń	ALL
GET	/work-orders/:id	Szczegóły zlecenia	ALL
POST	/work-orders	Utworzenie zlecenia	ALL
PATCH	/work-orders/:id	Edycja zlecenia	ADMIN, MANAGER
PATCH	/work-orders/:id/status	Zmiana statusu zlecenia	ADMIN, MANAGER, TECHNICIAN
PATCH	/work-orders/:id/assign	Przypisanie serwisanta	ADMIN, MANAGER
POST	/work-orders/:id/bhp-confirm	Potwierdzenie BHP	TECHNICIAN
GET	/work-orders/:id/messages	Lista wiadomości	ALL
POST	/work-orders/:id/messages	Dodanie wiadomości	ALL
GET	/work-orders/:id/parts	Lista użytych części	ALL
POST	/work-orders/:id/parts	Dodanie części do zlecenia	ADMIN, MANAGER, TECHNICIAN

- inventory — magazyn**

Metoda	Ścieżka	Opis	Role
GET	/inventory/categories	Lista kategorii	ALL
POST	/inventory/categories	Dodanie kategorii	ADMIN, MANAGER
DELETE	/inventory/categories/:id	Usunięcie kategorii	ADMIN
GET	/inventory/parts	Lista części (filtr po kategorii)	ALL
GET	/inventory/parts/low-stock	Części poniżej progu	ALL
GET	/inventory/parts/:id	Szczegóły części	ALL
POST	/inventory/parts	Dodanie części	ADMIN, MANAGER, WAREHOUSE

Metoda	Ścieżka	Opis	Role
PATCH	/inventory/parts/:id	Edycja części	ADMIN, MANAGER, WAREHOUSE
DELETE	/inventory/parts/:id	Usunięcie części	ADMIN, MANAGER
PATCH	/inventory/parts/:id/stock	Korekta stanu magazynowego	ADMIN, MANAGER, WAREHOUSE
GET	/inventory/loans	Lista aktywnych wypożyczeń	ADMIN, MANAGER
GET	/inventory/loans/my	Moje wypożyczenia	ALL
POST	/inventory/loans	Wypożyczenie narzędzia	ALL
PATCH	/inventory/loans/:id/return	Zwrot narzędzia	ALL

- preventive** — przeglądy prewencyjne

Metoda	Ścieżka	Opis	Role
GET	/preventive	Lista aktywnych planów	ALL
GET	/preventive/upcoming	Nadchodzące przeglądy	ALL
GET	/preventive/check	Ręczne uruchomienie harmonogramu	ADMIN
GET	/preventive/:id	Szczegóły planu	ALL
POST	/preventive	Utworzenie planu	ADMIN, MANAGER
PATCH	/preventive/:id	Edycja planu	ADMIN, MANAGER
DELETE	/preventive/:id	Usunięcie planu	ADMIN
POST	/preventive/:id/trigger	Ręczne wyzwolenie przeglądu	ADMIN, MANAGER

- notifications** — powiadomienia

Metoda	Ścieżka	Opis	Role
GET	/notifications	Moje powiadomienia	ALL
GET	/notifications/unread-count	Liczba nieprzeczytanych	ALL
GET	/notifications/all	Wszystkie powiadomienia	ADMIN
PATCH	/notifications/:id/read	Oznacz jako przeczytane	ALL
PATCH	/notifications/read-all	Oznacz wszystkie jako przeczytane	ALL
DELETE	/notifications/:id	Usunięcie powiadomienia	ALL
POST	/notifications	Wyślij powiadomienie do użytkownika	ADMIN, MANAGER
POST	/notifications/announcement	Ogłoszenie do wszystkich	ADMIN

- **dashboard** — panel główny

Metoda	Ścieżka	Opis	Role
GET	/dashboard? period=week month year	Dane KPI dla wybranego okresu	ALL

- **monitoring** — logi systemowe

Metoda	Ścieżka	Opis	Role
GET	/monitoring/requests	Logi requestów HTTP	ADMIN
GET	/monitoring/errors	Logi błędów aplikacji	ADMIN

8. Testy

Technologie

Do testowania obu warstw aplikacji wykorzystano **Vitest** — framework testowy kompatybilny z ekosystemem Vite, oferujący szybkie wykonanie testów i natywne wsparcie dla TypeScript. Frontend testowany jest dodatkowo przy użyciu **React Testing Library**.

Narzędzie	Zastosowanie
Vitest	Framework testowy (backend i frontend)
React Testing Library	Testowanie komponentów React
jsdom	Symulacja środowiska przeglądarki w testach frontendu

Testy backendu

Podejście:

Testy backendu podzielone są na dwa poziomy dla każdego modułu:

- **Testy routes** — weryfikują zachowanie endpointów HTTP: poprawne kody odpowiedzi, walidację danych wejściowych przez Zod, kontrolę dostępu opartą na rolach oraz propagację błędów z serwisu
- **Testy service** — weryfikują logikę biznesową w izolacji: operacje na bazie danych (mockowane przez Vitest), reguły domenowe, obsługę błędów i przypadki brzegowe

Zewnętrzne zależności (Prisma, Redis, jose) mockowane są przy użyciu `vi.mock()`, co pozwala testować logikę bez połączenia z rzeczywistą bazą danych.

Co jest testowane?

- Poprawne odpowiedzi dla prawidłowych danych wejściowych (happy path)
- Błędy walidacji (400) dla nieprawidłowych danych

- Odmowa dostępu (401, 403) dla brakujących lub niewystarczających uprawnień
- Błędy biznesowe (404, 409) gdy zasób nie istnieje lub wystąpił konflikt
- Logika domenowa — stany zleceń, korekty stanów magazynowych, potwierdzenie BHP, wersjonowanie dokumentów, obliczenia MTTR/MTBF, automatyczne powiadomienia

Testy frontendu

Podejście:

Testy frontendu sprawdzają komponenty i strony w izolacji przy użyciu React Testing Library. Wywołania API mockowane są przez `vi.mock()` lub `vi.spyOn()`. Środowisko przeglądarki symulowane jest przez jsdom.

Co jest testowane?

- Poprawne renderowanie elementów DOM
- Stany ładowania i błędów podczas komunikacji z API
- Interakcje użytkownika — wypełnianie formularzy, otwieranie modali, filtrowanie, wyszukiwanie
- Logika autoryzacji — przekierowanie niezalogowanych użytkowników, widoczność elementów w zależności od roli
- Obsługa odpowiedzi 401 (wygaśnięcie sesji)

Pokrycie modułów:

Obszar	Przetestowane komponenty
Komponenty wspólne	Header, Navbar, Sidebar, Footer, SearchBar, FilterPanel, AuthBtn
Auth	Login, Register, Logout
Dashboard	Strona główna, layout, przełącznik okresu
Inventory	Strona, modal dodawania/edycji, panel kategorii, alerty niskiego stanu
Locations	Strona, szczegóły, modal dodawania
Machines	Strona, szczegóły, modal dodawania/edycji, tabela dokumentów
Monitoring	Strona, tabela logów, modal szczegółów, pasek statystyk
Notifications	Strona
Preventive	Strona, szczegóły, modal
Users	Strona, szczegóły, modal dodawania/edycji, modal certyfikatów

Obszar	Przetestowane komponenty
Work-orders	Strona, szczegóły, modal

Uruchamianie testów

```
# Testy backendu
cd apps/api
npm run test

# Testy frontendu
cd apps/web
npm run test
```

9. Instrukcja uruchomienia

Pobranie projektu

```
git clone https://github.com/stafecki/cmms.git
cd cmms
npm install
```

Skopiuj i uzupełnij zmienne środowiskowe

```
cp .env.example .env
```

Uruchomienie aplikacji

Polecenie buduje i uruchamia wszystkie kontenery — bazy danych, backend i frontend:

```
npm run docker:up
```

Uruchom migracje:

```
npm run docker:migrate
```

Załaduj dane testowe:

```
npm run docker:seed
```

Dostęp do aplikacji

Zasób	Adres
Aplikacja webowa	<code>http://localhost:8080</code>
Backend API	<code>http://localhost:3000</code>

Domyślne konto administratora po seedowaniu bazy danych dostępne jest w pliku `apps/api/prisma/seed.ts`.

Zatrzymanie aplikacji

```
npm run docker:down
```